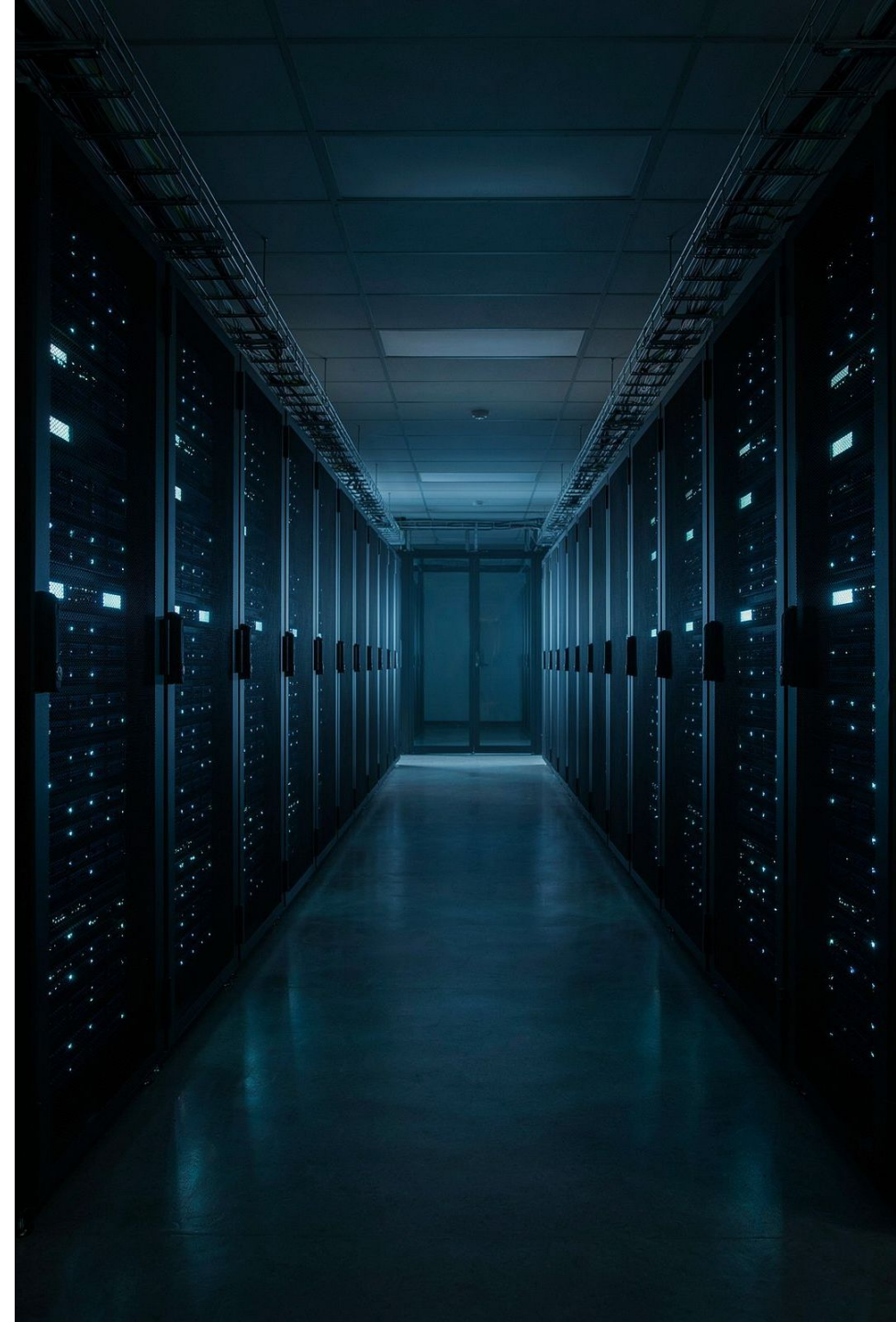


S11.2 · Transfer Learning y Fine-Tuning

Aprende cómo reutilizar el conocimiento de modelos entrenados en millones de imágenes para resolver tus propios problemas con pocos datos y menor coste computacional.

SESIÓN 11.2

APRENDIZAJE AUTOMÁTICO



Índice de contenidos

01

La idea de reutilizar conocimiento

Por qué entrenar desde cero ya no es siempre la mejor opción.

03

Extracción de características

Usar la red como un extractor fijo de representaciones.

05

Datasets pequeños y buenas prácticas

Estrategias cuando los datos etiquetados escasean.

02

Modelos preentrenados

ImageNet y los grandes repositorios de pesos disponibles.

04

Fine-tuning

Qué capas descongelar y cómo hacerlo correctamente.

06

Ejemplo paso a paso

Implementación completa con código y decisiones razonadas.



CONCEPTO CLAVE

¿Por qué reutilizar conocimiento?

Entrenar una red neuronal profunda desde cero requiere **millones de ejemplos etiquetados** y semanas de cómputo en GPUs de alto rendimiento. En la mayoría de proyectos reales, no disponemos de esos recursos. El **transfer learning** parte de una idea intuitiva: si un modelo ya aprendió a reconocer bordes, texturas, formas y objetos en un dataset enorme, ese conocimiento es transferible a tareas relacionadas. Es como contratar a un experto con años de experiencia en lugar de formar a alguien desde cero.



El transfer learning reduce drásticamente el tiempo de entrenamiento y mejora el rendimiento cuando los datos propios son limitados.

El paralelismo del experto humano

Aprendizaje desde cero

Un estudiante que empieza de cero necesita aprender todas las bases: leer, escribir, conceptos generales... antes de poder especializarse. Lento y costoso.

Transfer learning

Un profesional con años de experiencia aplica conocimiento previo a un nuevo problema. Solo necesita adaptarse a los detalles específicos del nuevo contexto.

Las redes neuronales funcionan igual: las **capas iniciales** aprenden representaciones genéricas (válidas para muchas tareas) y las **capas finales** aprenden representaciones específicas del dominio.

ImageNet y los grandes repositorios

ImageNet es el dataset de referencia en visión por computador: más de **1,2 millones de imágenes** distribuidas en 1.000 categorías. Entrenar una red sobre él durante días produce pesos que capturan una comprensión visual extraordinariamente rica del mundo.

Hoy existen repositorios públicos con miles de modelos listos para usar:

- **TensorFlow Hub:** modelos oficiales de Google.
- **PyTorch Hub:** ResNet, EfficientNet, ViT y más.
- **Hugging Face:** visión, texto, audio y multimodal.
- **timm:** la biblioteca más completa para visión.

Arquitecturas populares

VGG16 · VGG19

ResNet-50 · ResNet-101

InceptionV3 · Xception

EfficientNet B0–B7

Vision Transformer (ViT)

ConvNeXt

¿Qué aprende cada capa?

Las capas de una CNN preentrenada en ImageNet desarrollan una jerarquía visual que va de lo general a lo específico.



Capas finales

Partes de objetos específicos, caras, ruedas, texto. Muy dependientes del dominio.



Capas intermedias

Patrones complejos: esquinas, gradientes de color, estructuras repetidas.



Capas iniciales

Detectores de bordes, colores y texturas básicas.
Universales y transferibles.

📌 Cuanto más similares sean la tarea origen y la tarea destino, más capas podemos reutilizar sin modificar.

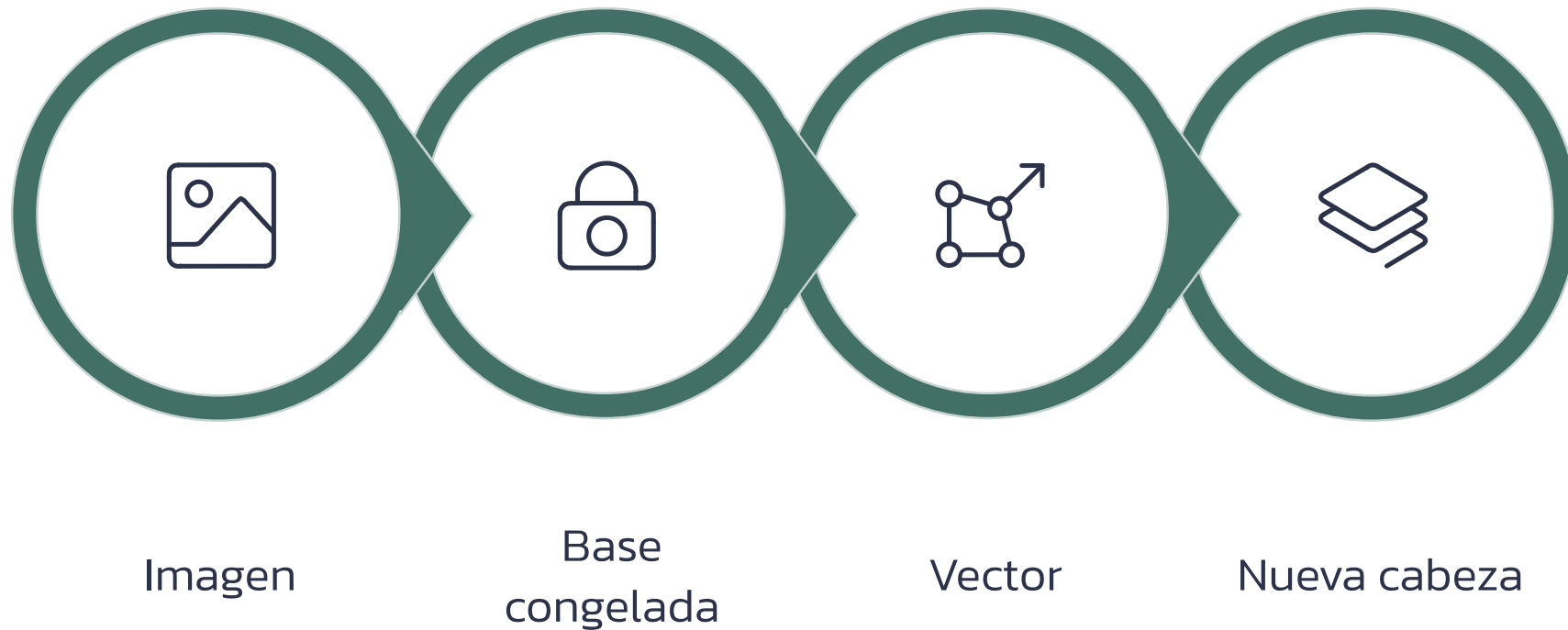


ESTRATEGIA 1

Extracción de características

En la **extracción de características** (feature extraction) usamos la red preentrenada como un **extractor fijo**: congelamos todos sus pesos y solo entrenamos una nueva cabeza de clasificación añadida al final. La red actúa como una función de transformación que convierte imágenes crudas en vectores de características ricos y significativos.

Extracción de características: el flujo completo



Las capas congeladas no actualizan sus gradientes durante el entrenamiento, lo que hace el proceso **muy rápido** y permite usar CPUs convencionales para la fase de entrenamiento de la cabeza.

Cuándo usar extracción pura de características



Dataset muy pequeño

Menos de 1.000 imágenes por clase. El fine-tuning podría sobreajustarse; es más seguro congelar toda la base.



Dominio similar al origen

Si tus imágenes se parecen a ImageNet (fotografías naturales), las características son directamente aplicables.



Recursos limitados

Sin GPU potente o con tiempo muy ajustado, extraer características precomputadas y entrenar un clasificador lineal es la opción más práctica.

Fine-tuning: ajuste fino de la red

El **fine-tuning** va un paso más allá: después de añadir la nueva cabeza de clasificación, **descongelamos parte de la base convolucional** y continuamos el entrenamiento con una tasa de aprendizaje muy baja. Así, los pesos ya aprendidos se ajustan sutilmente a las peculiaridades de nuestro nuevo dominio.

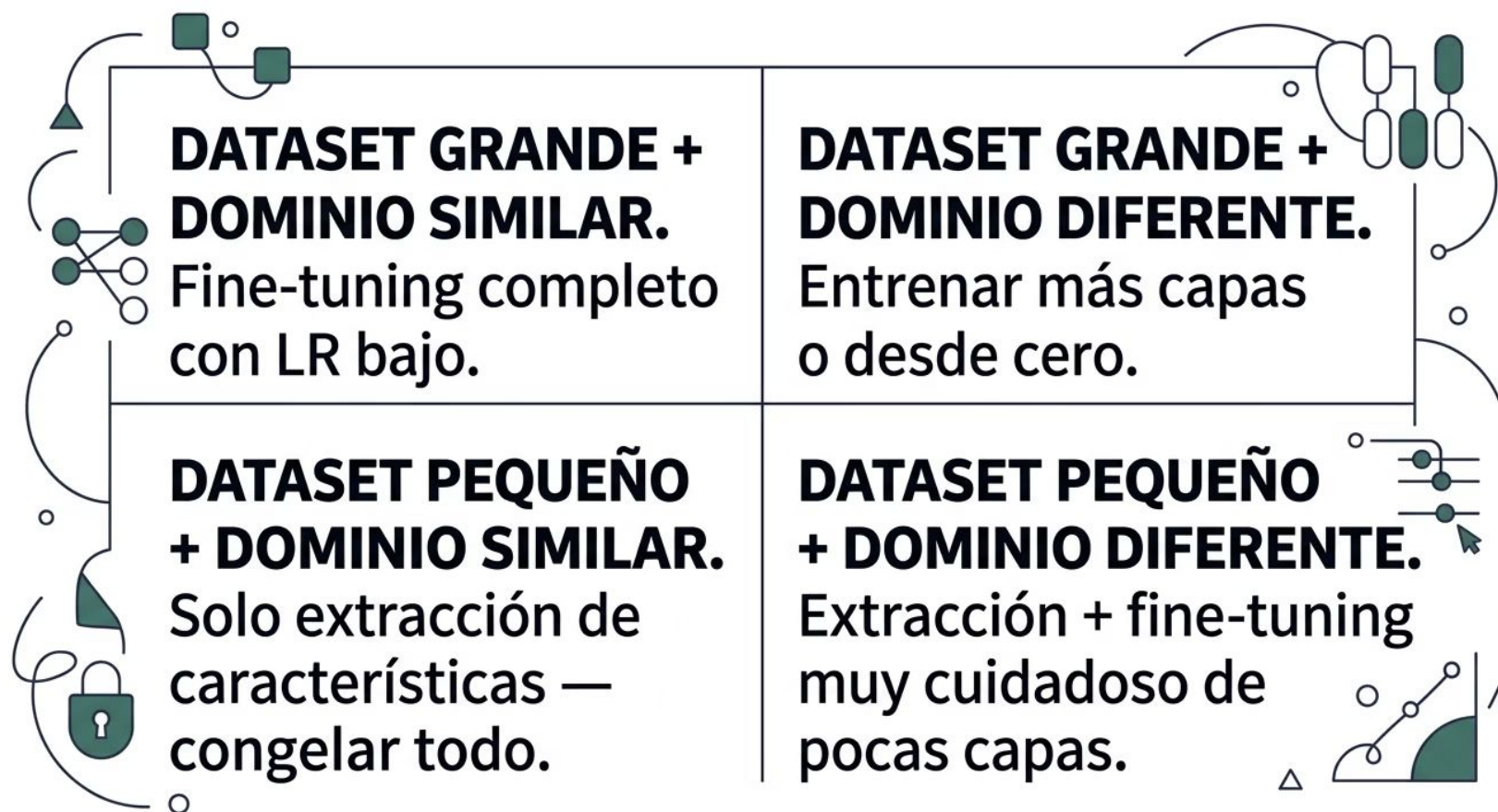
El objetivo es que las capas superiores de la red refinan sus representaciones para capturar las características específicas de nuestra tarea, sin destruir el conocimiento general ya adquirido.

Regla de oro

Empieza siempre entrenando **solo la cabeza** durante unas épocas. Luego desconfela gradualmente las capas superiores de la base. Nunca descongelar todo de golpe.

¿Qué capas descongelar?

La decisión depende del **tamaño del dataset** y de la **similitud con el dominio origen**. Esta matriz de decisión es el mapa esencial del transfer learning.

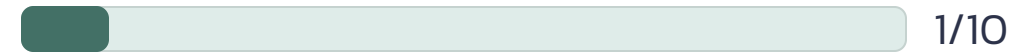


Tasa de aprendizaje en fine-tuning

La **tasa de aprendizaje** es el parámetro más crítico durante el fine-tuning. Debe ser **10 a 100 veces menor** que la usada en un entrenamiento normal. Una LR demasiado alta destruye los pesos preentrenados — fenómeno conocido como *catastrophic forgetting*.

- LR típica en entrenamiento normal: **1e-3**
- LR recomendada para fine-tuning: **1e-4 a 1e-5**
- LR diferencial por capas: más alta en capas nuevas, más baja en capas antiguas.

Tasas de aprendizaje diferenciales



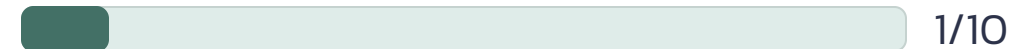
Cabeza nueva

LR completa (1e-3)



Capas superiores

LR reducida (1e-4)

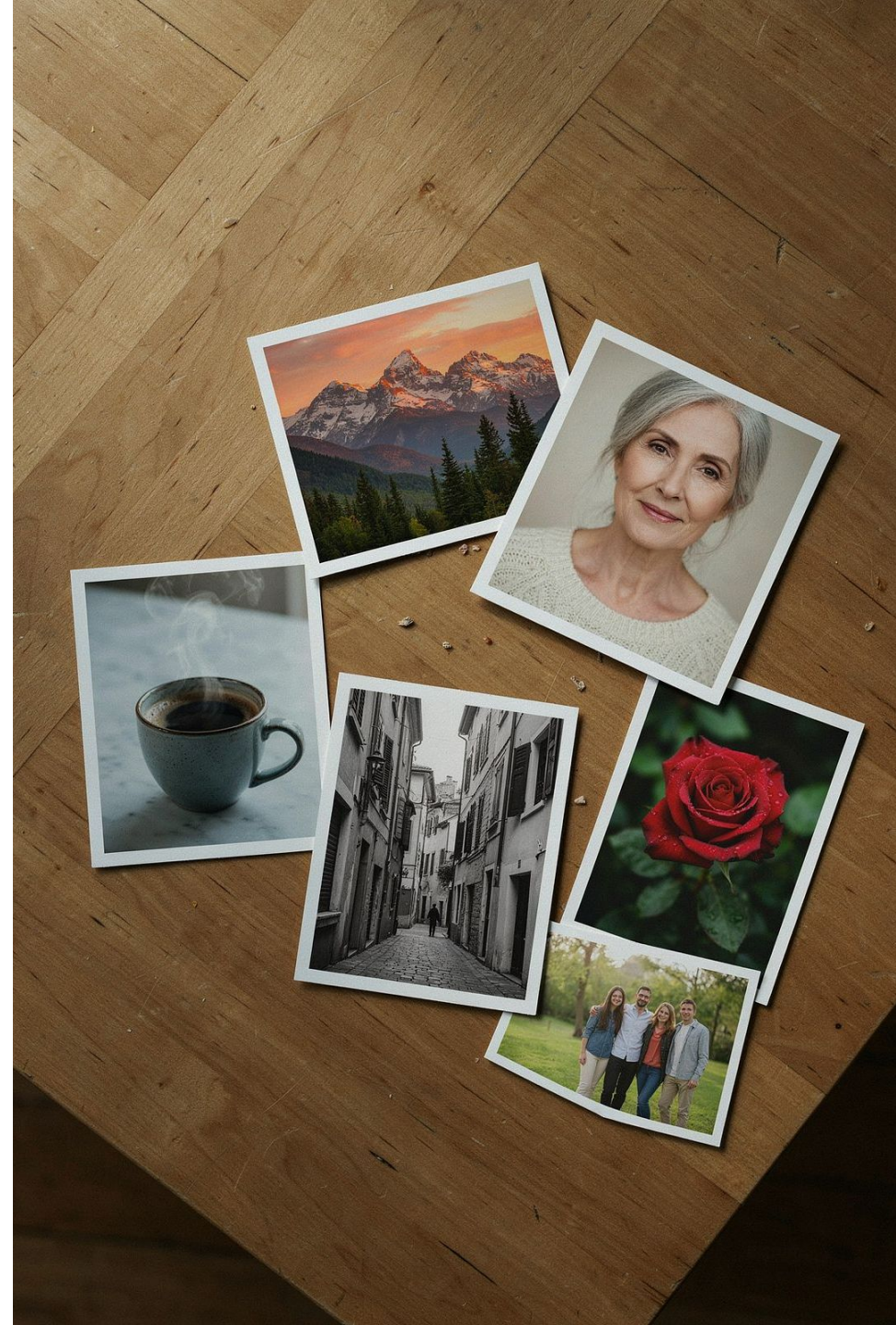


Capas inferiores

LR muy baja (1e-5)

Trabajar con datasets pequeños

En la práctica, muchos proyectos reales cuentan con **cientos, no millones**, de imágenes etiquetadas. El transfer learning es especialmente valioso en este escenario, pero requiere precauciones adicionales para evitar el sobreajuste.



Estrategias para datasets pequeños

Data augmentation agresivo

Rotaciones, recortes aleatorios, volteos horizontales, variaciones de brillo y color. Keras y PyTorch incluyen pipelines listos para usar. Técnicas avanzadas: Mixup, CutMix, RandAugment.

Regularización fuerte

Dropout (0.3–0.5) antes de la capa de clasificación final. Weight decay (L2) en el optimizador. Early stopping monitorizando la pérdida de validación.

Precomputar características

Pasar todo el dataset por la base congelada una sola vez y guardar los vectores. Entrenar después solo el clasificador: rápido y eficiente en memoria.

Modelos más pequeños

Con pocos datos, arquitecturas ligeras como EfficientNet-B0 o MobileNetV3 generalizan mejor que modelos gigantes con decenas de millones de parámetros.

Buenas prácticas: resumen esencial

1 Congela primero, afina después

Entrena la nueva cabeza hasta convergencia antes de descongelar cualquier capa de la base. Evita destruir representaciones valiosas desde el inicio.

3 Monitoriza la validación siempre

El overfitting puede aparecer rápido con datasets pequeños. Guarda el mejor checkpoint por métrica de validación, no por época.

2 Usa LR scheduler

Reduce la tasa de aprendizaje progresivamente durante el fine-tuning. ReduceLROnPlateau o Cosine Annealing son opciones probadas.

4 Normaliza con las estadísticas del preentrenamiento

Usa exactamente los mismos valores de media y desviación típica con los que se entrenó el modelo base (p. ej. ImageNet: mean=[0.485,0.456,0.406]).

Clasificación de razas de perros con EfficientNet

Vamos a construir un clasificador de **5 razas de perros** usando transfer learning sobre EfficientNet-B0 preentrenado en ImageNet. Disponemos de **500 imágenes por clase** — un dataset pequeño típico de un proyecto real.

Dataset

2.500 imágenes · 5 clases · Split
80/10/10

Arquitectura

EfficientNet-B0 +
GlobalAveragePooling + Dense(5,
softmax)

Estrategia

Fase 1: congelar base · Fase 2: fine-
tuning de últimas 20 capas

Paso 1: Cargar el modelo y preparar la cabeza

```
import tensorflow as tf

# Cargar base preentrenada SIN la cabeza de clasificación
base_model = tf.keras.applications.EfficientNetB0(
    weights='imagenet',
    include_top=False,    # Excluye la capa Dense de 1000 clases
    input_shape=(224, 224, 3)
)
base_model.trainable = False # Congelar todos los pesos

# Construir nueva cabeza
inputs = tf.keras.Input(shape=(224, 224, 3))
x = base_model(inputs, training=False)
x = tf.keras.layers.GlobalAveragePooling2D()(x)
x = tf.keras.layers.Dropout(0.4)(x)
outputs = tf.keras.layers.Dense(5, activation='softmax')(x)

model = tf.keras.Model(inputs, outputs)
```

El parámetro `training=False` en la llamada a la base es crucial: garantiza que las capas de Batch Normalization se comporten en modo inferencia aunque el modelo esté en fase de entrenamiento.

Paso 2: Entrenamiento de la cabeza y fine-tuning

```
# FASE 1: Entrenar solo la cabeza
model.compile(
    optimizer=tf.keras.optimizers.Adam(1e-3),
    loss='categorical_crossentropy',
    metrics=['accuracy']
)
model.fit(train_ds, epochs=10, validation_data=val_ds)

# FASE 2: Fine-tuning — descongelar últimas 20 capas
base_model.trainable = True
for layer in base_model.layers[:-20]:
    layer.trainable = False # Las anteriores siguen congeladas

model.compile(
    optimizer=tf.keras.optimizers.Adam(1e-5), # LR 100x menor
    loss='categorical_crossentropy',
    metrics=['accuracy']
)
model.fit(train_ds, epochs=20, validation_data=val_ds,
    callbacks=[tf.keras.callbacks.EarlyStopping(
        patience=5, restore_best_weights=True)])
```

Resultados esperados del ejemplo

Sin transfer learning

~54%

Accuracy val.

Red entrenada desde cero
con 2.500 imágenes.

50+

Épocas necesarias

Para converger
mínimamente.

Con transfer learning

~91%

Accuracy val.

EfficientNet-B0 fine-
tuneado.

30

Épocas totales

Fase 1 + Fase 2
combinadas.

El salto de ~37 **puntos porcentuales** en accuracy con el mismo dataset ilustra el enorme impacto práctico del transfer learning. El modelo preentrenado aporta un punto de partida incomparablemente superior.

Conclusiones clave

- Reutilizar es más inteligente que reinventar
Los modelos preentrenados condensan millones de horas de cómputo. Aprovechalos siempre como punto de partida antes de plantearte entrenar desde cero.
- Elige la estrategia según tus datos
Extracción de características para datasets muy pequeños o dominios similares; fine-tuning progresivo cuando tienes más datos o mayor distancia de dominio.
- La LR lo es todo en fine-tuning
Usa tasas 10–100 veces menores que en entrenamiento normal. Congela primero, afina después. Monitoriza siempre la validación.
- Con pocos datos, el transfer learning marca la diferencia
En proyectos reales con cientos de imágenes, la diferencia entre transfer learning y entrenamiento desde cero puede ser de décadas de puntos de accuracy.

